NeRF^2 Revisited: Cross-Platform Reproduction, Synthetic Benchmarking, and Efficiency

Ned Bitar
CSE 274: Image-Based Rendering
University of California, San Diego
Winter 2026

***Abstract*—Neural Radiance Fields (NeRF) have demonstrated that a continuous volumetric scene function can be learned from a sparse set of images, enabling novel-view synthesis [1]. NeRF^2 extends this idea from visible light to radio frequency (RF) propagation, treating received signal strength indication (RSSI) as the radiance and learning a spatial attenuation field through which rays are integrated [5]. However, the original implementation is wired to CUDA and evaluated only on limited datasets, leaving its generalization and computational efficiency under-studied.**

This paper makes three contributions. First, we reproduce NeRF^2 on Apple Silicon and confirm a median RSSI prediction error of 2.88 dBm on the BLE dataset at 10,000 iterations. Second, we introduce a synthetic benchmarking suite built with NVIDIA Sionna, consisting of three single-wall scenes with systematically varied material properties — concrete, metal, and wood — each matching the scale of the original dataset (6,000 TX positions, 21 gateways). NeRF^2 trained for 5,000 iterations achieves a mean squared error of 2.09 on concrete, 2.17 on metal, and 3.31 on wood. NeRF^2 is able to generalize to environments with different material properties. Third, we test four Instant-NGP accelerations and find that voxel grid resolution affects whether occupancy-grid skipping helps: a 192^3 grid on an RTX 3080 matches baseline accuracy within 0.10 dBm while training 2.7–5.6% faster. We discuss why the remaining Instant-NGP speedups do not carry over to radio.

***Index Terms*—neural radiance fields, RF propagation, BLE RSSI, Sionna, Instant-NGP, Apple Silicon, ray marching, indoor localization**

I. Introduction

In 2020, Mildenhall et al. introduced Neural Radiance Fields (NeRF) [1], a method that represents a 3-D scene as a continuous function mapping position and view direction to color and density. The camera ray is integrated along $F_\Theta : (\mathbf{x}, \mathbf{d}) \to (\mathbf{c}, \sigma)$, which allows NeRF to synthesize photo-realistic novel views from a sparse set of images. This result was a landmark contribution to image-based rendering and inspired hundreds of variants. NeRF in the Wild [2] expanded the method to unconstrained photographs. Nerfies [3] handled moving scenes. Instant-NGP [8] replaced positional encoding with a multi-resolution hash grid, cutting training time from hours to seconds.

The authors of NeRF^2 [5] make the observation that the physics of radio-frequency propagation follows the same volumetric integration as visible light. A Bluetooth Low Energy (BLE) signal travels from a transmitter, is attenuated by materials it passes through, and arrives at a receiver as the sum of all ray paths through the scene. In NeRF^2 we replace color with RSSI and density with RF attenuation. The model learns a spatial model of the radio environment from RSSI snapshots and achieves roughly 2.8 dBm median prediction error on a real indoor dataset of 42,000 measurements across 21 gateways.

Despite this, there are three gaps in the current state of the work. The first is accessibility — the implementation is tied to CUDA and will not even import on Apple Silicon. The second is benchmarking — the paper provides only one BLE dataset from one building, making it difficult to understand what drives accuracy or where the model breaks down. The third is efficiency — the original NeRF^2 uses a heavy 8-layer, 256-wide MLP and no one has tested whether acceleration techniques from Instant-NGP apply to radio.

This paper addresses all three gaps. Our contributions are:

- **Reproducibility on Apple Silicon.** We identify and fix bugs in the original codebase and confirm the published accuracy on Apple Silicon using the MPS backend. This required rewriting the renderer and portions of the model. Due to the relatively simple MLP structure, the port was straightforward once the device-allocation bugs were resolved.
- **Sionna Synthetic Benchmark.** We use NVIDIA Sionna's ray-tracing engine to generate three indoor BLE datasets with different wall materials and measure how accuracy changes as the training set shrinks.
- **Acceleration Study.** We implement and test four techniques from Instant-NGP: multi-resolution hash encoding, MLP architecture reduction, occupancy-grid skipping, and a fused Metal compute kernel. We report what works, what does not, and explain why. To do this we had to rebuild NeRF^2 from the ground up.

II. Related Work

Neural Radiance Fields. Neural Radiance Fields changed how we think about 3-D scene representation and proved that a neural network can learn a continuous scene function from images alone. Extensions pushed this further: NeRF in the Wild [2] handled photos taken in varying lighting and weather conditions, and Nerfies [3] handled moving objects. Instant-NGP [8] dramatically improved efficiency by replacing positional encoding with a multi-resolution hash grid, cutting training from hours to seconds. NeRF-GS [4] takes a hybrid

approach, combining NeRF's continuous representation with 3-D Gaussian splatting to address limitations around spatial awareness and sensitivity to initialization.

NeRF for RF Propagation. NeRF^2 [5] is the paper this work builds on directly. It adapts volumetric rendering to radio wave propagation and trains on BLE, RFID, and MIMO channel measurements. RFScape [6] takes a different approach: instead of learning a field of attenuation values, it learns the electromagnetic material properties of each object in the scene, which allows the model to be used in simulators such as NVIDIA Sionna. NeWRF [7] applies radiance fields to CSI reconstruction using a sparse set of channel measurements as training input. The gap this work fills is benchmarking and efficiency, which none of these papers address.

III. Background

A. NeRF^2 Model

NeRF^2 has two networks. An *attenuation MLP* F_a takes a 3-D position $\mathbf{x}$ with sinusoidal positional encoding at 10 frequency bands and outputs an attenuation coefficient σ. A *signal MLP* F_s also takes the transmitter position $\mathbf{p}_{tx}$ and view direction $\mathbf{d}$, and outputs a signal amplitude s. The received power is the integral of those contributions along the ray from transmitter to receiver:

$$\hat{P}_{rx} = \int_{t_n}^{t_f} T(t)\,\sigma(\mathbf{r}(t))\,s(\mathbf{r}(t), \mathbf{d}, \mathbf{p}_{tx})\,\mathrm{d}t \tag{1}$$

where $T(t) = \exp\!\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(u))\,\mathrm{d}u\right)$ is the accumulated transmittance, approximated with $S = 32$ stratified samples per ray.

B. Dataset

The BLE RSSI dataset [5] was collected in a 15,000 ft^2 office building. A single BLE transmitter was moved to 6,000 positions, with 21 fixed gateways recording RSSI at each location. This gives a $6{,}000 \times 21$ measurement matrix spanning -95 to -35 dBm. We use an 80/20 train/test split: 33,600 training samples and 8,400 test samples.

IV. Contribution 1: Reproduction and MPS Portability

A. Bugs Fixed

The original codebase was designed with CUDA in mind and would not work on Apple Silicon without crashing. We resolved these issues by porting the MLP and renderer to Apple Silicon. There were also missing files and issues with the default configuration that had to be addressed. One issue was the positional encoding for the transmitter position, which was turned off in the original config. With it disabled, the network receives raw XYZ coordinates spanning the scene and cannot distinguish between closely spaced transmitter positions. Fixing it was necessary for the model to converge. After all fixes, the model trains and evaluates on Apple Silicon's MPS backend.

B. Baseline Results

We trained the NeRF^2 model for 10,000 and 30,000 iterations on the provided BLE dataset. The results are shown in Table I.

TABLE I: Baseline NeRF^2 on the BLE RSSI test set (8,400 samples, 21 gateways).

Run	Median Error	Train Time
10k iter, MPS (ours)	2.88 dBm	7,179 s
10k iter, RTX 3080 (ours)	2.81 dBm	600 s
30k iter, MPS (ours)	2.80 dBm	∼21,000 s
Paper reported [5]	≈2.8 dBm	—

Our results match the published numbers across both hardware platforms. The RTX 3080 is about 12× faster to train than the MPS backend, which shows the cost of running this on consumer hardware.

We also tested how accuracy changes when the training set is smaller. At 1% of the data (about 336 training samples), the error is 7.3 dBm. It drops steadily and flattens out near 3.0 dBm above 60% of the data. NeRF^2 needs dense coverage of the environment to work well — sparse measurements are not enough to capture the RSSI variation in the scene.

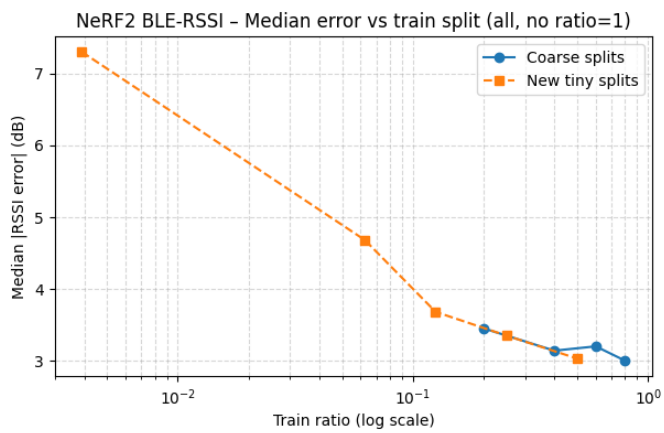

Fig. 1: Median error vs. training set size on the real BLE dataset.

V. Contribution 2: Synthetic Benchmarking with Sionna

A. Motivation

The real BLE dataset comes from one building under one set of conditions. There is no ground-truth propagation model, so we cannot test how the model responds to specific physical factors like wall material or multipath complexity. A physics-based simulator allows us to control the environment, letting us design experiments to test NeRF^2 in ways that real data cannot.

B. Simulation Pipeline

We used NVIDIA Sionna [9], a ray-tracing framework that simulates electromagnetic wave propagation from first principles. The pipeline works as follows:

1) Design a scene in Blender and export it as a Mitsuba XML file.
2) Load the scene into Sionna and configure it for the BLE carrier frequency ($f_c = 2.4$ GHz, 2 MHz bandwidth) with a single-element dipole antenna at TX and RX.
3) Place 6,000 TX positions and 21 RX positions in a uniform pattern over the scene, matching the real dataset layout.
4) Run the path solver with maximum ray depth 5, with line-of-sight, specular reflections, and refraction enabled.
5) Convert path amplitudes to received power in dBm: $P_{rx} = 10\log_{10}\left(\sum_k |a_k|^2 \cdot 10^3\right)$.
6) Export to NeRF^2 format.

The goal is to produce datasets that are structurally identical to the real BLE data, which means we can run NeRF^2 on them without any changes to the training code.

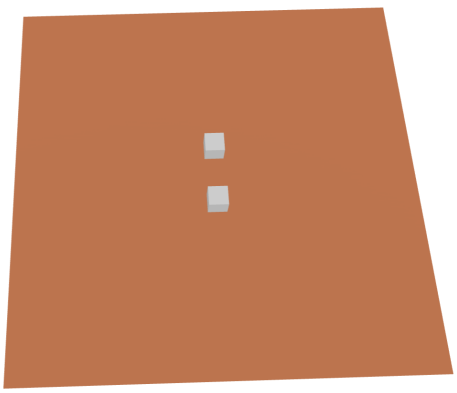

Fig. 2: Blender scene used in the Sionna pipeline. Each surface is assigned electromagnetic material properties to simulate concrete, metal, or wood at 2.4 GHz.

C. Material Comparison

To test whether NeRF^2 can adapt to different physical environments, we generated three datasets using a single-wall scene. For these tests the scene geometry, TX/RX layout, and data split (80/20) were kept identical. We trained NeRF^2 for 5,000 iterations on each dataset. The spatial RSSI maps in Fig. 3 capture the large-scale structure of the radio field in all three cases. Table II gives the quantitative breakdown.

Concrete and metal both behave as strong specular reflectors at 2.4 GHz. Their high conductivity produces well-defined reflection paths with predictable amplitudes. The model handles this well, achieving MSE below 2.2 on both materials. The model had more trouble with wood. Its lower permittivity means a significant portion of the signal passes through the wall rather than reflecting off it. This creates a more complex multipath structure that the model needs to encode. The result is an MSE of 3.31, which is 60% higher than concrete. This is not a failure of the model, but it does show that wood's frequency-dependent response is at the edge of what the MLP can represent within the 5,000 iteration budget.

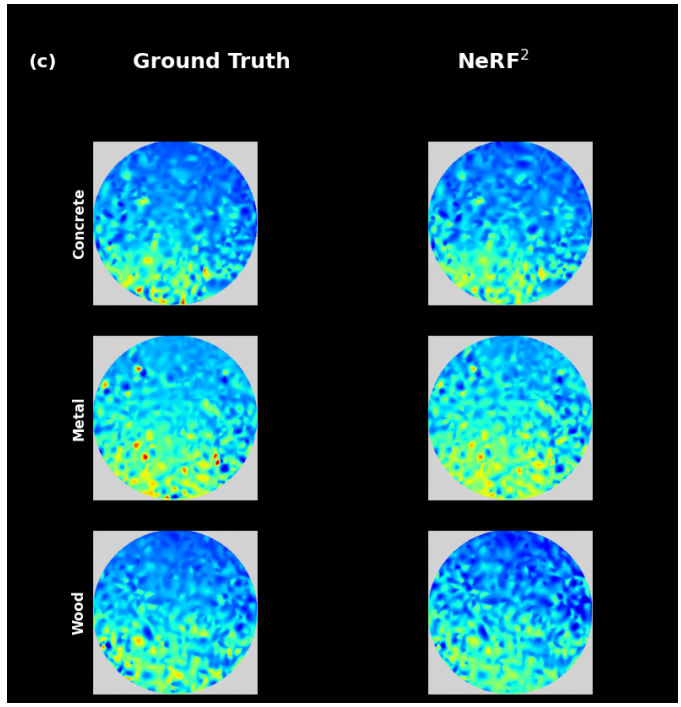

Fig. 3: Ground truth (left) vs. NeRF^2 prediction (right) for concrete (top), metal (middle), and wood (bottom).

TABLE II: NeRF^2 on Sionna single-wall scenes (5,000 iterations).

Wall Material	MSE (RSSI)
Concrete	2.087
Metal	2.169
Wood	3.306

VI. Contribution 3: Acceleration Study

We implement and test four acceleration techniques drawn from Instant-NGP [8]. Access to an NVIDIA GPU was limited to roughly 3 hours total across all experiments. Most RTX 3080 runs are capped at 300 iterations, and several comparisons that would ideally run to 30,000 iterations were not feasible in the available time.

A. Hash Encoding

Design. We replace the 60-dimensional sinusoidal positional encoding with a multi-resolution hash grid encoder at $L{=}8$ resolution levels, $F{=}2$ features per level, and a hash table size of $T{=}2^{16}$, giving a 16-dimensional feature per point. We also shrink the MLPs: the attenuation branch drops from 8×256 to 2×64, and the signal branch from 2×256 to 1×64.

Results. We ran two experiments. The first isolates the encoding while keeping the MLP identical to the baseline. The second is the full fast model, which also shrinks the MLP. We ran four hash encoder variants, all at 300 iterations on the RTX 3080 with the same 8×256 MLP. The variants differ only in the encoder implementation and learning rate.

V1 — Original. For the first attempt we set up the encoder to interpolate the 8 voxel corners to get the values stored at each corner. Instead of doing it in a vectorized way, the code looped through the 8 corners step by step. For each corner, the model fetched a learned feature vector from a hash table — one embedding lookup per corner per level. This was 2.3× slower than the baseline (83 s vs. 39 s) and reached 7.07 dBm.

V2 — Vectorized. For this attempt we rewrote the encoder to batch all 8 corners into a single index tensor before the embedding lookup. Speed dropped to 43.6 s, nearly matching the baseline. Error improved to 5.74 dBm. Same hash-table learning rate as V1.

Lite — Smaller hash tables. For this attempt we used smaller hash tables: 4 levels instead of 8, and table size 2^{14} instead of 2^{16}. The goal was to reduce the parameter count from 1.71M to 792k. Speed stayed at 43.8 s but accuracy got worse at 6.06 dBm.

V3 — Lower learning rate. Same vectorized encoder and full 8-level grid as V2, but the hash-table LR was lowered from 1e-2 to 5e-3. This was the best result: 5.29 dBm in 42.1 s.

Table III summarizes the ablation. Even the best hash-only variant (V3) is 0.78 dBm worse than baseline at the same iteration count.

TABLE III: Hash-only ablation (300 iter, RTX 3080). Full 8×256 MLP kept identical to baseline throughout.

Variant	Error	Total (s)	Change
Baseline (sinusoidal PE)	4.51 dBm	39.2	—
V1 (Python loop, lr=1e-2)	7.07 dBm	83.0	—
V2 (vectorized, lr=1e-2)	5.74 dBm	43.6	impl. speedup
Lite (4 levels, lr=5e-3)	6.06 dBm	43.8	smaller grid
V3 (vectorized, lr=5e-3)	**5.29 dBm**	**42.1**	lower LR

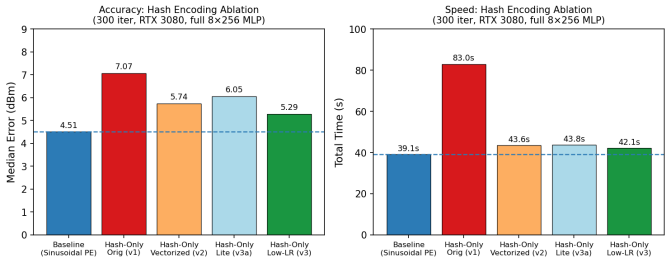

Fig. 4: Accuracy and total time for each hash encoder variant at 300 iterations (RTX 3080). Vectorizing the encoder recovers the speed regression; V3 gives the best accuracy.

Full fast model (hash encoding + smaller MLP). We also trained a version that replaces both the encoding and the MLP. At 300 training iterations, the fast model achieves 6.30 dBm versus 7.64 dBm for the baseline — it converges faster early on. But at 10,000 iterations, the baseline reaches 2.88 dBm while the fast model only reaches 6.63 dBm. Compared to the hash-only V3 result (5.29 dBm), shrinking the MLP accounts for roughly 1.3 dBm of additional error on top of the 0.78 dBm already lost from the encoding swap. More training does not close the gap. Table IV shows the full architecture comparison.

TABLE IV: Full fast model: architecture and 10k results (MPS).

	Baseline	Fast Model
Point encoding	Sinusoidal PE, 60-d	Hash grid, 16-d
Attenuation MLP	8×256	2×64
Signal MLP	2×256	1×64
MLP parameters	687,492	15,620
Hash table params	—	1,048,576
Train time (10k)	7,179 s	8,587 s
Eval time	12.5 s	117.4 s
Median error	2.88 dBm	6.63 dBm

Finest resolution sweep. To understand how the hash grid's spatial resolution affects radio prediction, we ran the vectorized hash encoder (V3 config) at four different finest-level resolutions — 128, 256, 512, and 1024 voxels — keeping the full 8×256 MLP. Table V and Fig. 5 show the results.

TABLE V: Hash encoder finest resolution sweep (full 8×256 MLP, RTX 3080).

Variant	Train (s)	Eval (s)	Median Error
Baseline (sinusoidal PE)	67.45	11.35	3.41 dBm
Hash-128	81.45	11.77	3.95 dBm
Hash-256	83.45	10.68	5.72 dBm
Hash-512	80.28	10.71	4.46 dBm
Hash-1024	80.00	10.63	4.56 dBm

The hash configurations do not improve consistently with finer resolution. Hash-128 gives the best result and stays closest to the baseline, suggesting that a relatively coarse grid is enough to capture the path-loss structure. Moving to Hash-256 actually hurts performance significantly, even though the spatial resolution increases. This shows that simply refining the grid does not guarantee better results. Hash-512 and Hash-1024 recover some of that lost performance but still do not surpass Hash-128. Overall, there is a tradeoff: coarser hashes miss fine detail, but finer hashes introduce additional difficulty without improving accuracy.

B. Architecture Reduction

We tested three smaller versions of the baseline to find a better speed–accuracy balance. All runs used 300 iterations on the real BLE dataset (Table VI).

The `slim d6×192` variant is the clear winner. It trains 1.9× faster with only 0.04 dBm extra error — essentially free speed. The `tiny d4×128` is 3.6× faster but adds 0.83 dBm. Whether that trade makes sense depends on the application.

C. Occupancy-Grid Skipping

Instant-NGP uses a binary occupancy grid to mark empty voxels and skip them during ray marching. We implemented

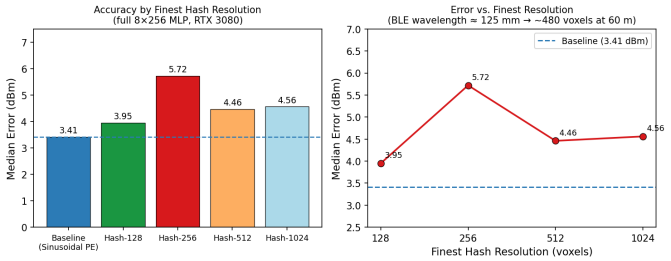

Fig. 5: Median error by finest hash resolution (left) and error vs. resolution on a $\log_2$ axis (right). The sinusoidal PE baseline is shown as a dashed line.

TABLE VI: Architecture sweep (300 iterations, BLE RSSI).

Variant	Params	Train	Error	Speedup
base d8×256	687k	215 s	6.66 dBm	1.0×
slim d6×192	325k	115 s	6.70 dBm	1.9×
slim d6×128	155k	72 s	7.52 dBm	3.0×
tiny d4×128	122k	60 s	7.49 dBm	3.6×

this and ran a sweep across seven voxel resolutions on an NVIDIA RTX 3080 with an occupancy threshold of 0.01 and 300 training iterations. No-grid baseline: 4.51 dBm in 40.7 s.

Six of the seven resolutions we tested are worse than running without a grid at all. The only one that works is 192^3: it hits 4.61 dBm in 39.6 s, matching baseline accuracy and training 2.7% faster. Every other resolution degrades accuracy by at least 0.69 dBm. Table VII and Fig. 6 show the full sweep.

TABLE VII: Voxel resolution sweep (RTX 3080, 300 iterations). No-grid baseline: 4.51 dBm, 40.7 s.

Voxel Res.	Train (s)	Total (s)	Error	vs. Baseline
16^3	26.2	37.8	7.10 dBm	+2.59
32^3	26.6	38.3	5.20 dBm	+0.69
64^3	28.7	39.4	6.17 dBm	+1.66
96^3	27.9	39.8	6.47 dBm	+1.96
128^3	28.1	40.0	6.24 dBm	+1.73
192^3	**28.5**	**39.6**	**4.61 dBm**	**+0.10**
256^3	29.0	41.0	6.57 dBm	+2.06

We then ran both the baseline and 192^3 out to 10,000 iterations on the RTX 3080. The 192^3 grid trains 2.7–5.6% faster at every checkpoint but never fully catches the baseline in accuracy. At 10,000 iterations the baseline is at 2.81 dBm and 192^3 is at 3.23 dBm. The gap narrows from 0.10 dBm at 300 iterations to 0.42 dBm at 10,000, but does not close.

D. Fused Metal Integration Kernel

We implemented a custom Metal compute shader that fuses 13 sequential ray-marching operations into one kernel dispatch. Each thread processes one ray over S=64 samples. On MPS, the fused kernel ran at 129.4 ms per batch versus 127.0 ms for PyTorch. The custom renderer without Metal ran at 124.5 ms, a 1.02× improvement. Accuracy was preserved throughout, with the maximum output difference between Metal and PyTorch in the range of 10^{-6}.

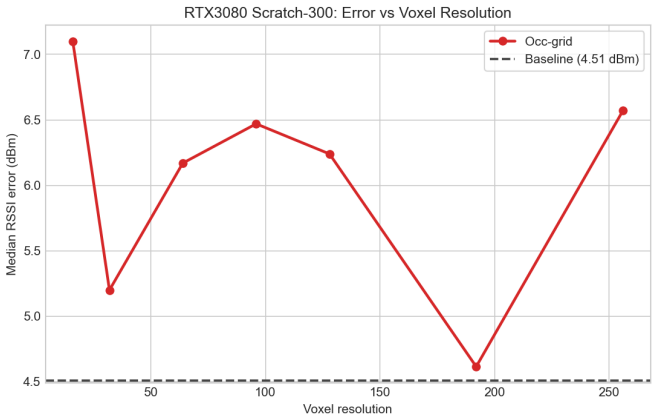

Fig. 6: Error vs. voxel resolution on the RTX 3080 (300 iterations). Only 192^3 matches baseline accuracy.

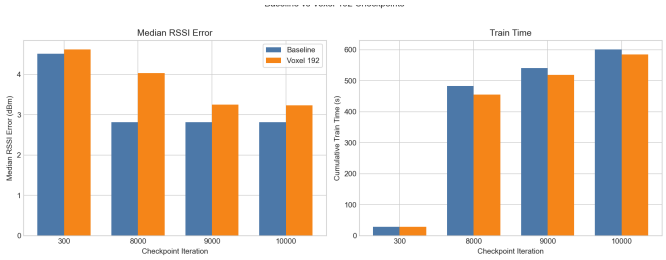

Fig. 7: Baseline vs. 192^3 grid across training checkpoints on the RTX 3080.

VII. Discussion

Each of the four Instant-NGP techniques hits a different wall when applied to radio.

Hash grid resolution and the BLE wavelength. The BLE wavelength at 2.4 GHz is 125 mm. Across the 60 m scene that corresponds to about 480 voxels. The resolution sweep shows there is no clean answer here. Hash-256 is the worst result in the sweep, even though it has twice the spatial resolution of Hash-128. At 256 voxels the grid sits at half the wavelength scale — fine enough to try to represent interference patterns but not fine enough to resolve them cleanly, so it fits noise instead of signal structure. Hash-128 stays coarser and just learns the large-scale path-loss gradient, which it does well. Hash-512 and Hash-1024 recover some performance by pushing below the wavelength scale, but they plateau around 4.5 dBm and do not improve further. Sinusoidal positional encoding at 10 frequency bands handles all spatial scales simultaneously, including sub-wavelength detail. A single fixed-resolution hash grid cannot do that. This is why even the best hash-only variant is 0.78 dBm worse than baseline, and why shrinking the MLP on top of that pushes the full fast model to 6.63 dBm.

Radio fills all of space. Occupancy-grid skipping works in visual NeRF because most of the scene is empty. In a radio scene, every voxel contributes: air introduces free-space path loss, walls reflect and absorb, corridors guide diffraction. The voxel sweep confirms this — six out of seven resolutions are

worse than running with no grid at all. The one exception is 192^3. At that resolution the grid identifies genuinely low-contribution regions without discarding voxels that carry real attenuation. Coarser grids lump different attenuation regimes together and skip things they should not. Finer grids add overhead without a matching benefit. At 10,000 iterations the gap is 0.42 dBm, which more training continues to shrink but does not eliminate.

Architecture reduction is the exception. Cutting from 8×256 to 6×192 gives $1.9\times$ faster training with only 0.04 dBm extra error. This works because the slim network has enough capacity for the task at this training budget. The baseline is oversized for what the BLE dataset requires at 300 iterations.

MPS kernel launch overhead. The original hash encoder and the Metal integration kernel both ran into the same problem: MPS dispatch overhead. Instant-NGP's hash encoder on CUDA is a single fused kernel. Our Python implementation fires 64 separate MPS launches per forward pass. The cost of those dispatches eats the savings from a smaller MLP. The Metal shader fuses 13 ray-marching operations but still cannot offset its own dispatch overhead at 32–64 samples per ray. On CUDA, both of these would likely show real gains.

VIII. Conclusion

NeRF^2 works. It predicts BLE RSSI within 2.8 dBm of ground truth and runs on Apple Silicon once the original bugs are fixed. The Sionna benchmark shows it generalizes to controlled physical environments with different wall materials.

Concrete and metal are handled well — the model learns their predictable reflection paths within 5,000 iterations. Wood is harder. Its lower permittivity means more signal passes through the wall than reflects off it, creating a richer multipath structure that the model has not converged on at 5,000 iterations. More training would close that gap.

For acceleration, architecture reduction is the most practical result: the slim 6×192 network trains $1.9\times$ faster with almost no accuracy cost. The occupancy grid at 192^3 saves a few percent in training time on CUDA but opens a 0.42 dBm gap at 10,000 iterations. Hash encoding does not work cleanly at this scale — the BLE wavelength requirements mean no single fixed resolution dominates, and the best we found (Hash-128) still sits 0.54 dBm above the sinusoidal PE baseline. The Metal kernel did not help on MPS.

Most GPU experiments were capped at 300 iterations due to the 3-hour access limit. The trends are clear but the absolute numbers would shift with more training. The open question is whether any hash encoder variant can reach baseline accuracy at full convergence. The resolution sweep suggests Hash-128 might get close, but that was not tested.

Acknowledgements

The author used Claude Sonnet (Anthropic) as an assistant during this project for debugging code, formatting this LaTeX report, and generating figures from experimental data.

References

[1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," in *Proc. ECCV*, 2020. arXiv:2003.08934.

[2] R. Martin-Brualla, N. Radwan, M. S. M. Sajjadi, J. T. Barron, A. Dosovitskiy, and D. Duckworth, "NeRF in the Wild: Neural Radiance Fields for Unconstrained Photo Collections," in *Proc. CVPR*, 2021. arXiv:2008.02268.

[3] K. Park, U. Sinha, J. T. Barron, S. Bouaziz, D. B. Goldman, S. M. Seitz, and R. Martin-Brualla, "Nerfies: Deformable Neural Radiance Fields," in *Proc. ICCV*, 2021. arXiv:2011.12948.

[4] S. Fang, I.-C. Shen, T. Igarashi, Y. Wang, Z. Wang, Y. Yang, W. Ding, and S. Zhou, "NeRF Is a Valuable Assistant for 3D Gaussian Splatting," in *Proc. ICCV*, 2025. arXiv:2507.23374.

[5] X. Zhao, Z. An, Q. Pan, and L. Yang, "NeRF^2: Neural Radio-Frequency Radiance Fields," in *Proc. ACM MobiCom*, 2023. arXiv:2305.06118.

[6] X. Chen, Z. Feng, K. Qian, and X. Zhang, "Radio Frequency Ray Tracing with Neural Object Representation," arXiv:2411.18635, 2024.

[7] H. Lu, C. Vattheuer, B. Mirzasoleiman, and O. Abari, "NeWRF: A Deep Learning Framework for Wireless Radiation Field Reconstruction and Channel Prediction," arXiv:2403.03241, 2024.

[8] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant Neural Graphics Primitives with a Multiresolution Hash Encoding," *ACM Trans. Graph.*, vol. 41, no. 4, 2022. arXiv:2201.05989.

[9] J. Hoydis, F. Aït Aoudia, S. Cammerer, M. Nimier-David, N. Binder, G. Marcus, and A. Keller, "Sionna RT: Differentiable Ray Tracing for Radio Propagation Modeling," arXiv:2303.11103, 2023.